Notes on Other SMBO Models: Random Forests (SMAC) and TPE

Luis Mendez

Overview

The previous note (Bayesian optimization with a Gaussian process surrogate) ended by flagging the GP's weak spots: cubic cost in the number of observations, and poor handling of high-dimensional, categorical, or *conditional* hyperparameter spaces (e.g., "number of dense layers" gating whether "layer_3_units" is even meaningful). This note covers two alternative surrogates that keep the exact same SMBO loop — initialize, fit surrogate, maximize acquisition, evaluate, augment — and swap out only the surrogate and how the acquisition step is computed from it: random forests (the approach used by SMAC) and the Tree-structured Parzen Estimator (TPE). It closes with simulated annealing, a cheaper sequential heuristic that is not really SMBO at all, included here because it is offered by the same libraries as a lightweight alternative.

Recap: The SMBO Loop

All three methods below instantiate the loop from the previous note: evaluate a handful of points to bootstrap, fit a model to $\mathcal{D}_t = \{(\mathbf{x}_i, y_i)\}$, use that model to pick the next $\mathbf{x}_{t+1}$, evaluate the expensive objective, and repeat. What changes is only *what* is fitted to $\mathcal{D}_t$ and *how* the next point is chosen from it.

Bayesian Optimization with Random Forests (SMAC)

Why swap the surrogate

A GP models $p(y \mid \mathbf{x})$ through a kernel that measures similarity between points, which implicitly assumes a continuous space with a meaningful distance metric. Real hyperparameter spaces are often a mix of integers, unordered categories, and conditional structure, none of which a stationary kernel represents naturally. SMAC (Sequential Model-based Algorithm Configuration) replaces the GP with a random forest regressor, which splits on any variable type without needing a distance metric at all.

Random forest as a surrogate

Fit a random forest regression model to $\mathcal{D}_t$ in the usual way: each tree is grown on a bootstrap sample with random feature subsampling at each split. Two quantities are then read off the forest to stand in for the GP's $\mu(\mathbf{x})$ and $\sigma(\mathbf{x})$:

$$\mu(\mathbf{x}) = \frac{1}{T}\sum_{t=1}^{T} \hat{f}_t(\mathbf{x}), \qquad \sigma^2(\mathbf{x}) = \frac{1}{T}\sum_{t=1}^{T} \left(\hat{f}_t(\mathbf{x}) - \mu(\mathbf{x})\right)^2, \tag{1}$$

the mean and empirical variance of the individual trees' predictions. These are then plugged directly into the same EI, PI, or LCB formulas from the previous note — the acquisition machinery does not change, only the (μ, σ) that feed it.

It is worth being explicit that $\sigma(\mathbf{x})$ here is a *heuristic* disagreement-between-trees measure, not a calibrated posterior standard deviation the way the GP's is. It has no conjugacy argument behind it. In practice it still behaves the right way qualitatively (low where the forest has seen nearby points, higher where it has not), which is enough for the acquisition function to do its job.

Why random forests handle mixed and conditional spaces well

- **Categorical variables**: a tree splits a categorical feature into subsets directly; no kernel or embedding is required.
- **Conditional hyperparameters**: inactive parameters (e.g., a third convolutional layer's kernel size when only two layers are used) can simply be filled with a placeholder value; trees are robust to such structurally-irrelevant inputs in a way a kernel's distance computation is not.
- **Scaling**: fitting a forest is roughly linear in the number of observations per tree, versus the GP's $O(n^3)$ matrix inverse, so the surrogate itself stays cheap even as trials accumulate.
- **High dimensions**: random feature subsampling at each split makes forests comparatively robust as the number of hyperparameters grows, whereas a single shared length scale (or one per dimension, in ARD) becomes harder to fit well with few observations per dimension.

Implementation in Scikit-Optimize

`skopt.forest_minimize` mirrors `gp_minimize`'s interface exactly (`n_calls`, `n_initial_points`, `acq_func`, `random_state`), with a `base_estimator` of `'RF'` (random forest) or `'ET'` (extra trees); `gbrt_minimize` is the same idea with gradient-boosted trees as the surrogate instead, trading the forest's variance-across-trees uncertainty for one derived from quantile regression.

Worked example

Tuning a small CNN (2 convolutional blocks, a variable number of dense layers and nodes, ReLU/sigmoid activation, learning rate) on MNIST digit images with `forest_minimize`: the search space is defined with `skopt.space`'s `Integer`, `Real`, and `Categorical` dimensions exactly as in the GP note, the objective trains

the CNN and returns negative validation accuracy, and `plot_objective`/`plot_evaluations` are used afterward to inspect which hyperparameters the forest surrogate found most influential and in what order the space was explored. The resulting CNN reaches strong accuracy across all ten digit classes, confirming the forest surrogate is a workable drop-in even though its uncertainty is only a heuristic.

Pros / Cons

- **Pros**: handles categorical and conditional hyperparameters natively, no kernel to design, scales better than a GP as observations accumulate, robust in higher dimensions.
- **Cons**: tree predictions are piecewise-constant, so $\mu(\mathbf{x})$ and $\sigma(\mathbf{x})$ are step functions rather than smooth surfaces, which can make the acquisition function's optimum less precisely localized than under a GP; the forest's variance is a heuristic uncertainty proxy with no probabilistic guarantee behind it, unlike the GP's closed-form posterior.

Tree-structured Parzen Estimator (TPE)

The key idea: model $p(\mathbf{x} \mid y)$, not $p(y \mid \mathbf{x})$

Both the GP and the random forest surrogate are regression models: fit $p(y \mid \mathbf{x})$, then derive μ and σ at candidate points. TPE inverts this. Fix a threshold y^* (in practice the γ-quantile of observed scores so far, e.g. $\gamma = 0.15$, itself a hyperparameter of the algorithm) and split the observed pairs into two groups by whether they cleared it:

$$p(\mathbf{x} \mid y) = \begin{cases} \ell(\mathbf{x}) & y < y^* \quad \text{(the "good" points)} \\ g(\mathbf{x}) & y \geq y^* \quad \text{(the "bad" points),} \end{cases} \tag{2}$$

with $p(y < y^*) = \gamma$. Each of ℓ and g is a nonparametric density built from Parzen windows (kernel density estimation): every observed $\mathbf{x}_i$ in the group contributes a kernel centered at $\mathbf{x}_i$, and the density is their sum. "Tree-structured" refers to how this density is built over the search space itself: for a space with conditional hyperparameters (a choice node gating which child hyperparameters exist), ℓ and g are built hierarchically over that tree, only modeling a child hyperparameter's density using the trials where it was actually active. This is what lets TPE represent deeply nested spaces — exactly the kind Hyperopt's `hp.choice` is designed to express — without any special-casing.

Connection to Expected Improvement

This inverted model is not an ad hoc replacement for EI; Bergstra et al. (2011) show it computes the same quantity. Expanding $\mathrm{EI}_{y^*}(\mathbf{x})$ from the previous note using Bayes' rule to swap $p(y \mid \mathbf{x})$ for $p(\mathbf{x} \mid y)$, and using $p(\mathbf{x}) = \gamma\ell(\mathbf{x}) + (1-\gamma)g(\mathbf{x})$, yields

$$\mathrm{EI}_{y^*}(\mathbf{x}) \;\propto\; \left(\gamma + \frac{g(\mathbf{x})}{\ell(\mathbf{x})}(1-\gamma)\right)^{-1}, \tag{3}$$

which is monotonically increasing in $\ell(\mathbf{x})/g(\mathbf{x})$. So maximizing EI under this model is the same as maximizing the ratio of the "good" density to the "bad" density — a point is promising exactly when it looks like where good scores have come from and unlike where bad scores have come from.

Why this is convenient algorithmically

The GP and random forest both need an explicit inner optimization to maximize the acquisition function over $\mathcal{X}$. TPE sidesteps this: because $\ell(\mathbf{x})$ is built to be directly samplable (it is a mixture of kernels), the algorithm draws a batch of candidate points from $\ell(\mathbf{x})$ itself, scores each by $\ell(\mathbf{x})/g(\mathbf{x})$, and keeps the best-scoring candidate as $\mathbf{x}_{t+1}$. This avoids a separate global optimization step entirely, at the cost of only searching where ℓ already places mass rather than the whole space.

Implementation with Hyperopt

Hyperopt's `hp` module defines the search space (`hp.quniform`, `hp.uniform`, `hp.choice` for categorical/conditional branches), `tpe.suggest` is passed as the `algo` to `fmin`, and a `Trials` object records every evaluated configuration and its loss for later inspection, exactly as `skopt`'s result object does.

Worked example

The same MNIST CNN tuned with `forest_minimize` above is retuned with `fmin(algo=tpe.suggest)`: same search space (learning rate, layer counts, node counts, activation), same objective (negative validation accuracy). Because the search space is expressed with `hp.choice` nodes for the layer counts, the conditional structure (whether a fourth dense layer's node count is even sampled) falls out naturally from the tree-structured density, with no manual placeholder values needed the way the random forest surrogate required.

Pros / Cons

- **Pros**: naturally expresses nested/conditional search spaces, which is precisely what deeply branching spaces like neural architecture choices need; fitting kernel density estimates is cheap and needs no matrix inversion; candidate generation by direct sampling from $\ell(\mathbf{x})$ avoids a separate acquisition-maximization step.
- **Cons**: the default estimator treats hyperparameters independently when building ℓ and g, so it does not capture interactions between hyperparameters the way a jointly-fit GP or forest regression can; the quantile γ is itself a tuning choice; uncertainty is implicit in the density ratio rather than an explicit, interpretable $\sigma(\mathbf{x})$.

Simulated Annealing: A Non-Surrogate Sequential Heuristic

Hyperopt also offers `anneal.suggest`, described as "simulated annealing": it starts like random search and, as trials accumulate, narrows sampling toward the region around the best trials found so far, with the width of that neighborhood shrinking over the run. Unlike SMAC or TPE it fits no explicit probabilistic model of $\mathcal{D}_t$ — there is no likelihood, no posterior, no acquisition function being maximized, only a shrinking-neighborhood sampling rule anchored on the current incumbent. It is informed only in the loose sense that later samples depend on which earlier trials scored well, not through any surrogate.

Worked comparison

Tuning an XGBoost classifier on the breast cancer dataset with 30 evaluations each of `rand.suggest` (pure random search) and `anneal.suggest`: random search reached mean CV accuracy ≈ 0.972, simulated annealing ≈ 0.962. On this particular run, simulated annealing did not outperform pure random search. This is a useful negative result: with a budget this small, most of it is still spent in (or near) the initial exploratory phase, leaving little room for the narrowing behavior to pay off, and a single run says little about typical behavior given the added randomness of the annealing schedule itself. It is a reminder that "sequential" and "informed" do not automatically mean "better than random search" at small budgets — the SMAC and TPE results above are informed in a much stronger sense (an explicit model of the response surface) and are the methods worth reaching for when sample efficiency actually matters.

COMPARING THE SURROGATES

Method	Models	Uncertainty	Mixed/conditional spaces	Cost per iteration
GP (previous note)	$p(y \mid \mathbf{x})$	Closed-form posterior $\sigma(\mathbf{x})$	Poor without care	$O(n^3)$
Random forest (SMAC)	$p(y \mid \mathbf{x})$	Heuristic, variance across trees	Good	$\approx O(n \log n)$
TPE	$p(\mathbf{x} \mid y)$	Implicit, in density ratio	Good, esp. nested spaces	Cheap KDE fit
Simulated annealing	Neither	None	N/A — no model	Trivial

PRACTICAL GUIDANCE

- Prefer the **GP** for low-to-moderate dimensional, mostly continuous spaces with a modest evaluation budget, where its calibrated uncertainty is worth the cubic cost.
- Prefer **random forests (SMAC)** when the space mixes categorical and continuous hyperparameters, has some conditional structure, or has enough dimensions that a shared kernel becomes hard to trust.
- Prefer **TPE** for deeply nested/conditional spaces (architecture search, pipelines with many optional stages), where the tree structure of the space itself is the main difficulty.
- Treat **simulated annealing** (or plain random search) as the cheap baseline to beat, not as a substitute for a real surrogate-based search when evaluations are expensive enough that sample efficiency matters.
- All four are available behind near-identical interfaces (`skopt.gp_minimize`/`forest_minimize`/`gbrt_minimize`, `hyperopt.fmin` with `tpe.suggest`/`anneal.suggest`/`rand.suggest`), so switching between them in practice is mostly a one-line change plus adapting the search-space definition to the library's API.